

NATIONAL ECONOMICS UNIVERSITY (NEU)
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE & ARTIFICIAL INTELLIGENCE (FDA)

OFFICIAL SOLUTIONS & MARKING RUBRIC

Course: Fundamental Programming Concepts in Python (DSAI1003)

Progress Midterm Examination (60 Minutes) — Paper Code: 101

Weighting: 40% Quiz, 30% Fill-in, 30% Full Code | **Difficulty:** 80% Basic (80 pts), 20% Medium (20 pts)

Note for Instructors and Teaching Assistants: This document contains the official answer keys, conceptual explanations, reference Python implementations, sample outputs, and itemized grading rubrics for the 60-minute Midterm Exam. All grading must follow the rubrics consistently.

Section A: Multiple-Choice Quiz Solutions (40 points — 20 questions, 2 pts each)

Difficulty: 16 Basic questions (32 pts, 80%), 4 Medium questions (8 pts, 20%).

Part 1: Questions 1 to 10

Q#	Key	Level	Conceptual Explanation & Rule Reference	Marks
Q1	B	Basic	CPython compiles source into platform-independent bytecode (.pyc), executed by the Python Virtual Machine (PVM).	2 pts
Q2	C	Basic	Variable names cannot start with a digit. <code>2nd_quarter</code> raises a <code>SyntaxError: invalid decimal literal</code> .	2 pts
Q3	B	Basic	The built-in <code>input()</code> function unconditionally returns a string (<code>str</code>), requiring explicit type casting for arithmetic.	2 pts
Q4	A	Basic	<code>sep="->"</code> separates arguments; <code>end="#"</code> prevents newline. Next print appends immediately: <code>DSFE->2026#NEU</code> .	2 pts
Q5	B	Basic	Division by zero is syntactically valid but raises a runtime exception: <code>ZeroDivisionError</code> during execution.	2 pts
Q6	A	Medium	Precedence: <code>**</code> (highest) $\rightarrow$ <code>//</code> , <code>*</code> , <code>%</code> (left-to-right) $\rightarrow$ <code>+</code> . $5^2 = 25$; $17//4 = 4$; $4 \times 2 = 8$; $19\%25 = 19$; $8 + 19 = 27$.	2 pts
Q7	B	Basic	Decimal fractions (0.1, 0.2) cannot be stored exactly in IEEE 754 binary floating-point, causing roundoff error ($0.1 + 0.2 \neq 0.3$).	2 pts
Q8	B	Basic	Strings are immutable in Python. In-place index modification (<code>name[0] = "J"</code>) raises a <code>TypeError</code> .	2 pts
Q9	B	Basic	String replication: <code>"2"*3</code> is <code>"222"</code> ; <code>"4"*2</code> is <code>"44"</code> . String concatenation joins them into <code>"22244"</code> .	2 pts
Q10	B	Basic	Format specifier <code>:.2f</code> inside an f-string formats a float to fixed-point representation with exactly 2 decimal places.	2 pts

Part 2: Questions 11 to 20

Q#	Key	Level	Conceptual Explanation & Rule Reference	Marks
Q11	A	Medium	Slice [start:stop:step] with negative step traverses backwards. Indices 8, 6, 4, 2 yield 'n', 'i', 'S', 't' → "niSt".	2 pts
Q12	A	Basic	Single equals = is the assignment operator. Double equals == is the relational equality comparison operator.	2 pts
Q13	B	Basic	Chained comparison 10 < x <= 25 is evaluated as 10 < x and x <= 25, evaluating x only once.	2 pts
Q14	A	Basic	Precedence hierarchy for boolean operators is strictly: not (highest), followed by and, then or (lowest).	2 pts
Q15	B	Basic	s.isdigit() returns True if all characters are digits and length ≥ 1; returns False otherwise.	2 pts
Q16	A	Medium	De Morgan's Law: $\neg(A \wedge B) \equiv (\neg A) \vee (\neg B)$. Result: score < 50 or status == "probation".	2 pts
Q17	B	Basic	In if-elif-else, branches evaluate top-down. The first True condition (82 ≥ 50) executes and remaining branches are bypassed.	2 pts
Q18	A	Basic	In short-circuit A or B, if A is True, B is completely bypassed because the result is already known.	2 pts
Q19	A	Basic	Strings are compared lexicographically by Unicode code point. Lowercase 'b' (98) is greater than uppercase 'A' (65).	2 pts
Q20	D	Medium	In B: x != 0 is False, short-circuiting and. In C: x == 0 is True, short-circuiting or. Both avoid division by zero.	2 pts

Section A Summary:

- Basic: 16 Qs × 2 pts (Q1–Q5, Q7–Q10, Q12–Q15, Q17–Q19) = **32 points (80.0% of Section A)**
- Medium: 4 Qs × 2 pts (Q6, Q11, Q16, Q20) = **8 points (20.0% of Section A)**
- Section A Total: **40 points (40.0% of total exam)**

Section B: Coding — Fill-in Problems Solutions (30 points)

Difficulty: 24 points Basic (80%), 6 points Medium (20%).

Problem B1: Currency Exchange Teller Utility (12 points — 4 blanks, 3 pts each [All Basic])

```
exchange_rate, fee_rate, ref_code = 25450.0, 0.02, "TERM04-TX99281"
usd_amount = float(input("Enter exchange amount in USD: "))      # [B1.1] (3 pts - Basic)
base_vnd = usd_amount * exchange_rate
fee_vnd = base_vnd * fee_rate                                     # [B1.2] (3 pts - Basic)
total_vnd = base_vnd + fee_vnd
terminal_id = ref_code[:6]                                       # [B1.3] (3 pts - Basic)
print(f"Total Payable: {total_vnd:,.2f} VND")                   # [B1.4] (3 pts - Basic)
```

Blank	Level	Accepted Answer(s) & Grading Criteria	Marks
[B1.1]	Basic	float(input("Enter exchange amount in USD: ")) or float(input()) <i>Award 3 pts for correct float(input(...)). Deduct 1 pt if int() used.</i>	3 pts
[B1.2]	Basic	base_vnd * fee_rate or usd_amount * exchange_rate * fee_rate <i>Award 3 pts for valid fee formula.</i>	3 pts
[B1.3]	Basic	ref_code[:6] or ref_code[0:6] <i>Award 3 pts. Award 1.5 pts if off-by-one (e.g. [:7]).</i>	3 pts
[B1.4]	Basic	total_vnd:,.2f or total_vnd:,.2F <i>Award 3 pts for comma and 2 decimals. Award 1.5 pts if comma omitted (total_vnd:.2f).</i>	3 pts

Problem B2: Bank Loan Risk Tier Classifier (18 points — 6 blanks, 3 pts each)

```

credit_score = int(input("Enter Credit Score (300-850): "))
monthly_income = float(input("Enter Monthly Income (USD): "))
monthly_debt = float(input("Enter Monthly Debt (USD): "))
has_delinquency = input("Any Delinquency? (yes/no): ").strip().lower() == "yes"
has_guarantor = input("Guarantor available? (yes/no): ").strip().lower() == "yes"

# [B2.1] Basic: Validate credit_score is within bounds [300, 850]:
if not (300 <= credit_score <= 850):
    print("ERROR: Invalid credit score."); exit()

# [B2.2] Basic: Healthy Debt-to-Income (DTI) condition (monthly_debt / monthly_income <= 0.40):
is_dti_healthy = (monthly_debt / monthly_income) <= 0.40

# [B2.3] Basic: Tier 1 check: score >= 750 AND income >= 3000 AND NO delinquency:
tier1_eligible = credit_score >= 750 and monthly_income >= 3000 and not has_delinquency

card_tier = "DECLINED"
if tier1_eligible and is_dti_healthy:
    card_tier = "PLATINUM PRIME"

# [B2.4] Basic: Second tier check: score >= 650 AND income >= 1800:
elif credit_score >= 650 and monthly_income >= 1800:
    card_tier = "GOLD STANDARD"

# [B2.5] Medium: Short-circuit division guard: income > 0 AND debt ratio < 0.50:
safe_debt_check = monthly_income > 0 and (monthly_debt / monthly_income < 0.50)

# [B2.6] Medium: Fallback check: guarantor AND (score >= 600 OR income >= 2500) AND NOT delinquency:
if card_tier == "DECLINED" and safe_debt_check:
    if has_guarantor and (credit_score >= 600 or monthly_income >= 2500) and not has_delinquency:
        card_tier = "BASIC SECURED"
print(f"Decision: {card_tier}")

```

Blank	Level	Accepted Answer(s) & Grading Criteria	Marks
[B2.1]	Basic	300 <= credit_score <= 850 or credit_score >= 300 and credit_score <= 850 <i>Award 3 pts. Award 1.5 pts if strict inequality (<) used.</i>	3 pts
[B2.2]	Basic	(monthly_debt / monthly_income) <= 0.40 or monthly_debt / monthly_income <= 0.4 <i>Award 3 pts for correct ratio comparison.</i>	3 pts
[B2.3]	Basic	credit_score >= 750 and monthly_income >= 3000 and not has_delinquency <i>Also accept ... and has_delinquency == False. Deduct 1 pt if not omitted.</i>	3 pts
[B2.4]	Basic	credit_score >= 650 and monthly_income >= 1800 <i>Award 3 pts for correct combined condition in the elif header.</i>	3 pts
[B2.5]	Medium	monthly_income > 0 and (monthly_debt / monthly_income < 0.50) <i>Award 3 pts for proper short-circuit ordering (monthly_income > 0 MUST come before division). Deduct 2 pts if division placed first.</i>	3 pts
[B2.6]	Medium	has_guarantor and (credit_score >= 600 or monthly_income >= 2500) and not has_delinquency <i>Award 3 pts. Mandatory parentheses around (... or ...); deduct 1.5 pts if parentheses omitted due to operator precedence violation.</i>	3 pts

Section B Summary:

- Basic: [B1.1]–[B1.4] (12 pts) + [B2.1]–[B2.4] (12 pts) = **24 points (80.0% of Section B)**
- Medium: [B2.5] (3 pts) + [B2.6] (3 pts) = **6 points (20.0% of Section B)**
- Section B Total: **30 points (30.0% of total exam)**

Section C: Applied Full-Code Programming & Design Solutions (30 points)

Difficulty Distribution: 24 points Basic (80%), 6 points Medium (20%). Problems C1 and C2 share a common business scenario with two distinct requirements. Problem C3 evaluates conditional branching and string methods.

Problem Scenario for C1 & C2: University Banquet Event Dining Invoice Engine

A university banquet dining hall generates an itemized bill for student club dinner reservations and calculates equal payment splits among attendees based on the following specifications:

- **Inputs:** Food & beverage subtotal (subtotal in USD, float), number of dining guests (num_diners, int), and catering service charge rate (service_rate, float, e.g. 0.10 for 10%).
- **Financial Formulas:**
 - Service Charge = subtotal × service_rate
 - Taxable Subtotal = subtotal + Service Charge
 - Value Added Tax (8% VAT) = Taxable Subtotal × 0.08
 - Grand Total Bill = Taxable Subtotal + VAT
 - Per-Diner Payment Share = Grand Total Bill/num_diners

Problem C1: Algorithmic Specification & Pseudocode (8 points — [All Basic])

*Requirement: Formulate structured **pseudocode** only (e.g., using INPUT, COMPUTE / SET, PRINT, BEGIN, END). Do NOT write Python code.*

```
BEGIN
    PROMPT AND READ subtotal, num_diners, service_rate
    COMPUTE service_charge = subtotal * service_rate
    COMPUTE taxable_subtotal = subtotal + service_charge
    COMPUTE vat = taxable_subtotal * 0.08
    COMPUTE grand_total = taxable_subtotal + vat
    COMPUTE per_diner_share = grand_total / num_diners
    PRINT "Subtotal:", subtotal
    PRINT "Service Charge:", service_charge
    PRINT "VAT (8%):", vat
    PRINT "Grand Total:", grand_total
    PRINT "Per-Diner Share:", per_diner_share
END
```

Problem C2: Sequential Python Implementation (10 points — [All Basic])

*Requirement: Complete Python program following sequential **Input** → **Process** → **Output**. Strictly NO conditional statements (if-then) or string methods.*

```
# Step 1: Input collection with explicit numeric casting
subtotal = float(input("Enter food and beverage subtotal ($): "))
num_diners = int(input("Enter number of dining guests: "))
service_rate = float(input("Enter service rate (e.g. 0.10 for 10%): "))

# Step 2: Sequential calculations (strictly no branching)
service_charge = subtotal * service_rate
taxable_subtotal = subtotal + service_charge
vat = taxable_subtotal * 0.08
grand_total = taxable_subtotal + vat
per_diner_share = grand_total / num_diners

# Step 3: Formatted receipt display (2 decimals with $)
print(f"Subtotal: ${subtotal:.2f} | Service ({service_rate*100:.0f}%): ${service_charge:.2f} | VAT (8%): ${vat:.2f}")
print(f"GRAND TOTAL: ${grand_total:.2f} | EACH PERSON PAYS ({num_diners} diners): ${per_diner_share:.2f}")
```

Combined Grading Rubric for Problems C1 & C2 (18 points total)	Level	Marks
[C1] Input Specification: Correctly prompts and reads subtotal, num_diners, service_rate.	Basic	2.0 pts
[C1] Sequential Computation Flow: Accurately evaluates formulas in correct dependency order (service → taxable → VAT → grand total → per-diner share).	Basic	4.0 pts
[C1] Output Display & Conventions: Uses standard pseudocode keywords (BEGIN, END, READ, COMPUTE, PRINT) without Python syntax artifacts.	Basic	2.0 pts
[C2] Input Collection & Casting: Reads subtotal (float), num_diners (int), service_rate (float) with explicit typecasting.	Basic	2.5 pts
[C2] Sequential Processing: Accurately evaluates service charge, taxable amount, 8% VAT, grand total, and per-guest split.	Basic	4.5 pts
[C2] Formatted Receipt & Compliance: Uses f-strings with :.2f and \$. Strictly avoids any if/elif/else structures or string methods.	Basic	3.0 pts

Problem C3: Logistics Freight Shipping & Dimensional Weight Engine (12 points)

*Difficulty Breakdown: 6 points Basic + 6 points Medium. Note: This is the **ONLY** problem requiring string processing methods and conditional branching (if-elif-else).*

Reference Python Implementation:

```
# Step 1: Input Collection
pkg_id = input("Enter Package Tracking ID: ").strip()
weight_kg = float(input("Enter actual weight (kg): "))
length_cm = float(input("Enter package length (cm): "))
width_cm = float(input("Enter package width (cm): "))
height_cm = float(input("Enter package height (cm): "))
zone = input("Enter destination zone (DOMESTIC/INTERNATIONAL): ").strip().upper()

# Step 2: Input Validation with String Methods [4 pts - Basic]
is_id_valid = pkg_id.startswith("PKG-") and len(pkg_id) >= 8
is_dims_valid = weight_kg > 0 and length_cm > 0 and width_cm > 0 and height_cm > 0
is_zone_valid = zone in ("DOMESTIC", "INTERNATIONAL")

if not (is_id_valid and is_dims_valid and is_zone_valid):
    print("ERROR: Invalid package parameters.")
else:
    # Step 3: Dimensional & Billable Weight [1 pt - Basic]
    vol_weight = (length_cm * width_cm * height_cm) / 5000.0
    billable_weight = max(weight_kg, vol_weight)

    # Step 4: Tiered Progressive Base Freight [1 pt - Basic]
    if billable_weight <= 2.0:
        base_freight = 12.00
    elif billable_weight <= 10.0:
        base_freight = 12.00 + 3.50 * (billable_weight - 2.0)
    else:
        base_freight = 40.00 + 2.00 * (billable_weight - 10.0)

    # Step 5: International Customs Surcharge with $25.00 floor [6 pts - Medium]
    if zone == "INTERNATIONAL":
        intl_surcharge = max(0.30 * base_freight, 25.00)
    else:
        intl_surcharge = 0.00

    total_shipping = base_freight + intl_surcharge

    # Step 6: Formatted Manifest Output
    print("=====")
    print("LOGISTICS FREIGHT DISPATCH MANIFEST")
    print("=====")
    print(f"Package ID: {pkg_id} | Billable Weight: {billable_weight:.1f} kg")
    print(f"Base Freight Charge:    ${base_freight:6.2f} | International Surcharge: ${intl_surcharge:6.2f}")
    print("-----")
    print(f"TOTAL SHIPPING PAYABLE:    ${total_shipping:6.2f}")
    print("=====")
```

Sample Verification Test Runs:

Case 1: Standard Domestic Package (Volumetric weight dominates)

Input: PKG-88120, weight: 3.0 kg, dims: 40x30x20 cm, zone: DOMESTIC

→ vol_weight = 24000/5000 = 4.8 kg → billable = 4.8 kg

→ base_freight = 12.00 + 3.50 * 2.8 = \$21.80 | intl_surcharge = \$0.00 | Total = \$21.80

Case 2: International Express (Floor surcharge applies)

Input: PKG-77291, weight: 4.8 kg, dims: 30x20x10 cm, zone: INTERNATIONAL

→ billable = 4.8 kg → base = \$21.80

→ 0.30 × 21.80 = \$6.54 < \$25.00 → intl_surcharge = \$25.00 (minimum floor applied)

→ Total = \$21.80 + \$25.00 = \$46.80

Case 3: Validation Rejection

Input: PKG-12 (length < 8) → Displays: "ERROR: Invalid package parameters."

Problem C3: Itemized Marking Rubric (12 points — 6 Basic + 6 Medium)

Grading Component & Criteria	Level	Marks
1. Input Collection & Validation with String Methods: - Prompts and parses tracking code, weight, 3 dimensions, destination zone (1.0 pt) - Checks <code>startswith("PKG-")</code> and <code>len(pkg_id) >= 8</code> (1.5 pts) - Checks all dimensions and weight are strictly positive (> 0) (0.5 pt) - Validates zone with <code>zone.strip().upper()</code> in ("DOMESTIC", "INTERNATIONAL") (0.5 pt) - Prints specified error message and halts calculation on failure (0.5 pt)	Basic	4.0 pts
2. Dimensional Weight & Progressive Base Freight: - Computes $\text{vol_weight} = (L \times W \times H)/5000$ and $\text{billable_weight} = \max(\text{weight}, \text{vol})$ (1.0 pt) - Correct 3-tier progressive freight: tier 1 (≤ 2: \$12), tier 2 ($2 < w \leq 10$: $\\$12 + 3.5(w - 2)$), tier 3 ($> 10$: $\\$40 + 2(w - 10)$) (1.0 pt)	Basic	2.0 pts
3. International Customs Surcharge Logic (Medium): - Identifies international shipments and sets domestic surcharge to \$0.00 (1.5 pts) - Computes 30% surcharge: $0.30 \times \text{base_freight}$ (1.5 pts) - Strictly enforces the \$25.00 minimum surcharge floor using $\max(0.30 * \text{base}, 25.0)$ or equivalent condition (3.0 pts)	Medium	6.0 pts
Total for Problem C3	6 Basic + 6 Medium	12.0 pts

Common Student Misconceptions & Instructor Grading Notes for Section C:

- C1 (Pseudocode):** Penalize if students write actual Python code (deduct 3.0 pts). The goal of C1 is language-agnostic algorithmic design.
- C2 (Sequential Code):** Penalize if students use `if/else` branching (deduct 3.0 pts). All computations must be purely sequential.
- C3 (International Floor):** Many students calculate $0.30 \times \text{base_freight}$ but omit the $\geq \$25.00$ floor condition. Enforce full 3.0 pts deduction for this omission.

Overall Exam Assessment Matrix

Component	Basic (80%)	Medium (20%)	Total Points	Weight (%)
Section A: Multiple-Choice Quiz	32 pts (16 Qs)	8 pts (4 Qs)	40 pts	40%
Section B: Coding — Fill-in Problems	24 pts (8 blanks)	6 pts (2 blanks)	30 pts	30%
Section C: Applied Full-Code Programming	24 pts (C1: 8 + C2: 10 + C3: 6)	6 pts (C3 Medium)	30 pts	30%
Grand Total	80 pts (80.0%)	20 pts (20.0%)	100 pts	100.0%